

Exercise 3.1

$\lfloor (\text{value-of } \langle\langle x \rangle\rangle \rho) \rfloor = 10$

$\lfloor (\text{value-of } \langle\langle 3 \rangle\rangle \rho) \rfloor = 3$

$\lfloor (\text{value-of } \langle\langle v \rangle\rangle \rho) \rfloor = 5$

$\lfloor (\text{value-of } \langle\langle i \rangle\rangle \rho) \rfloor = 1$

Exercise 3.2

$val \in Bool \in ExpVal.$

Exercise 3.3

We are able to describe the arithmetic operations in terms of subtraction. We cannot do so if we chose addition.

Exercise 3.4

Let $\rho = [x=[33], y=[22]]$.

$$\frac{\frac{\text{(value-of-program } \langle\langle \text{if zero? } \neg(x, 11) \text{ then } \neg(y, 2) \text{ else } \neg(y, 4) \rangle\rangle)}{\text{(value-of } \langle\langle \text{if zero? } \neg(x, 11) \text{ then } \neg(y, 2) \text{ else } \neg(y, 4) \rangle\rangle \rho)}}{\frac{\text{(value-of } \langle\langle \text{zero? } \neg(x, 11) \rangle\rangle \rho) = (\text{bool-val } \#f)}}{\frac{\text{(value-of } \langle\langle \neg(y, 4) \rangle\rangle \rho)}}{[18]}}$$

Exercise 3.5

$$\frac{\begin{array}{l} \text{(value-of } \langle\langle \text{let } x = 7 \\ \quad \text{in let } y = 2 \\ \text{in let } y = \text{let } x = \neg(x, 1) \text{ in } \neg(x, y) \\ \quad \text{in } \neg(\neg(x, 8), y) \rangle\rangle \rho_0) \end{array}}{skibidi - too - hard}$$

Exercise 3.6

```
;; the-grammar
(expression
  ("minus" "(" expression ")")
  minus-exp)

;; value-of
(minus-exp (exp1)
  (num-val (- (expval->num (value-of exp1 env))))))
```

Exercise 3.7

```
;; the-grammar
(expression
  ("+" "(" expression "," expression ")")
  add-exp)
(expression
  ("*" "(" expression "," expression ")")
  mul-exp)
(expression
  ("/" "(" expression "," expression ")")
  div-exp)

;; value-of
(add-exp (exp1 exp2)
  (num-val (+ (expval->num (value-of exp1 env))
              (expval->num (value-of exp2 env))))))
(mul-exp (exp1 exp2)
  (num-val (* (expval->num (value-of exp1 env))
              (expval->num (value-of exp2 env))))))
(div-exp (exp1 exp2)
  (let ((num2 (expval->num (value-of exp2 env))))
```

```

(if (zero? num2)
    (eopl:error 'value-of "division by zero")
    (num-val (/ (expval->num (value-of exp1 env))
                num2)))))

```

Exercise 3.8

```

;; the-grammar
(expression
  ("equal?" "(" expression "," expression ")")
  equal?-exp)
(expression
  ("greater?" "(" expression "," expression ")")
  greater?-exp)
(expression
  ("less?" "(" expression "," expression ")")
  less?-exp)

;; value-of
(equal?-exp (exp1 exp2)
  (bool-val (= (expval->num (value-of exp1 env))
               (expval->num (value-of exp2 env)))))
(greater?-exp (exp1 exp2)
  (bool-val (> (expval->num (value-of exp1 env))
               (expval->num (value-of exp2 env)))))
(less?-exp (exp1 exp2)
  (bool-val (< (expval->num (value-of exp1 env))
               (expval->num (value-of exp2 env)))))

```

Exercise 3.9

ExpVal = Int + Bool + Pair + Empty – list

```

;; the-grammar

```

```

(expression
  ("cons" "(" expression "," expression ")")
  cons-exp)
(expression
  ("car" "(" expression ")")
  car-exp)
(expression
  ("cdr" "(" expression ")")
  cdr-exp)
(expression
  ("null?" "(" expression ")")
  null?-exp)
(expression
  ("emptylist")
  empty-list-exp)

;; value-of
(cons-exp (exp1 exp2)
  (pair-val (value-of exp1 env) (value-of exp2 env)))
(car-exp (exp1)
  (pair-car (expval->pair (value-of exp1 env))))
(cdr-exp (exp1)
  (pair-cdr (expval->pair (value-of exp1 env))))
(null?-exp (exp1)
  (cases expval (value-of exp1 env)
    (empty-list-val () (bool-val #t))
    (else (bool-val #f))))
(empty-list-exp () (empty-list-val))

;; expval
(pair-val
  (val1 expval?)

```

```

    (val2 expval?))
(empty-list-val)

;; expval->pair : ExpVal -> Pair
(define expval->pair
  (lambda (v)
    (cases expval v
      (pair-val (val1 val2)
        (lambda (b)
          (if b val1 val2)))
      (else (expval-extractor-error 'pair v)))))

;; pair-car : Pair -> ExpVal
(define pair-car
  (lambda (pair)
    (pair #t)))

;; pair-cdr : Pair -> ExpVal
(define pair-cdr
  (lambda (pair)
    (pair #f)))

```

Exercise 3.10

Expression ::= list(*{Expression}*^(*l*))

```

;; the-grammar
(expression
  ("list" "(" (separated-list expression ",") ")")
  list-exp)

;; value-of
(list-exp (exps)

```

```

(letrec
  ((value-of-list-exp
    (lambda (exps)
      (if (null? exps)
          (empty-list-val)
          (pair-val (value-of (car exps) env)
                     (value-of-list-exp (cdr exps))))))
  (value-of-list-exp exps)))

```

Exercise 3.11

```

(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))
      (var-exp (var) (apply-env env var))
      (diff-exp (exp1 exp2) (value-of-diff-exp exp1 exp2 env))
      (zero?-exp (exp1) (value-of-zero?-exp exp1 env))
      (if-exp (exp1 exp2 exp3) (value-of-if-exp exp1 exp2 exp3 env))
      (let-exp (var exp1 body) (value-of-let-exp var exp1 body env))
      (minus-exp (exp1) (value-of-minus-exp exp1 env))
      (add-exp (exp1 exp2) (value-of-add-exp exp1 exp2 env))
      (mul-exp (exp1 exp2) (value-of-mul-exp exp1 exp2 env))
      (div-exp (exp1 exp2) (value-of-div-exp exp1 exp2 env))
      (equal?-exp (exp1 exp2) (value-of-equal?-exp exp1 exp2 env))
      (greater?-exp (exp1 exp2) (value-of-greater?-exp exp1 exp2 env))
      (less?-exp (exp1 exp2) (value-of-less?-exp exp1 exp2 env))
      (cons-exp (exp1 exp2) (value-of-cons-exp exp1 exp2 env))
      (car-exp (exp1) (value-of-car-exp exp1 env))
      (cdr-exp (exp1) (value-of-cdr-exp exp1 env))
      (null?-exp (exp1) (value-of-null?-exp exp1 env))
      (emptylist-exp () (emptylist-val))
      (list-exp (exps) (value-of-list-exp exps env))))))

```

```

;; value-of-diff-exp : Exp * Exp * Env -> ExpVal
(define value-of-diff-exp
  (lambda (exp1 exp2 env)
    (let ((val1 (value-of exp1 env))
          (val2 (value-of exp2 env)))
      (let ((num1 (expval->num val1))
            (num2 (expval->num val2)))
        (num-val
         (- num1 num2))))))

;; value-of-zero?-exp : Exp * Env -> ExpVal
(define value-of-zero?-exp
  (lambda (exp1 env)
    (let ((val1 (value-of exp1 env)))
      (let ((num1 (expval->num val1)))
        (if (zero? num1)
            (bool-val #t)
            (bool-val #f))))))

;; value-of-if-exp : Exp * Exp * Exp * Env -> ExpVal
(define value-of-if-exp
  (lambda (exp1 exp2 exp3 env)
    (let ((val1 (value-of exp1 env)))
      (if (expval->bool val1)
          (value-of exp2 env)
          (value-of exp3 env)))))

;; value-of-let-exp : Identifier * Exp * Exp * Env -> ExpVal
(define value-of-let-exp
  (lambda (var exp1 body env)
    (let ((val1 (value-of exp1 env)))
      (value-of body
                 (env-extend env var val1)))))

```

```

                                (extend-env var val1 env))))))

;; value-of-minus-exp : Exp * Env -> ExpVal
(define value-of-minus-exp
  (lambda (exp1 env)
    (num-val (- 0 (expval->num (value-of exp1 env))))))

;; value-of-add-exp : Exp * Exp * Env -> ExpVal
(define value-of-add-exp
  (lambda (exp1 exp2 env)
    (num-val (+ (expval->num (value-of exp1 env))
                (expval->num (value-of exp2 env))))))

;; value-of-mul-exp : Exp * Exp * Env -> ExpVal
(define value-of-mul-exp
  (lambda (exp1 exp2 env)
    (num-val (* (expval->num (value-of exp1 env))
                (expval->num (value-of exp2 env))))))

;; value-of-div-exp : Exp * Exp * Env -> ExpVal
(define value-of-div-exp
  (lambda (exp1 exp2 env)
    (let ((divisor (expval->num (value-of exp2 env))))
      (if (zero? divisor)
          (eopl:error 'div-exp "division by zero")
          (num-val (/ (expval->num (value-of exp1 env))
                      divisor))))))

;; value-of-equal?-exp : Exp * Exp * Env -> ExpVal
(define value-of-equal?-exp
  (lambda (exp1 exp2 env)
    (bool-val (= (expval->num (value-of exp1 env))
                 (expval->num (value-of exp2 env)))))

```



```

                                (expval->num (value-of exp2 env))))))

;; value-of-greater?-exp : Exp * Exp * Env -> ExpVal
(define value-of-greater?-exp
  (lambda (exp1 exp2 env)
    (bool-val (> (expval->num (value-of exp1 env))
                  (expval->num (value-of exp2 env))))))

;; value-of-less?-exp : Exp * Exp * Env -> ExpVal
(define value-of-less?-exp
  (lambda (exp1 exp2 env)
    (bool-val (< (expval->num (value-of exp1 env))
                  (expval->num (value-of exp2 env))))))

;; value-of-cons-exp : Exp * Exp * Env -> ExpVal
(define value-of-cons-exp
  (lambda (exp1 exp2 env)
    (pair-val (value-of exp1 env) (value-of exp2 env))))

;; value-of-car-exp : Exp * Env -> ExpVal
(define value-of-car-exp
  (lambda (exp1 env)
    (car (expval->pair (value-of exp1 env)))))

;; value-of-cdr-exp : Exp * Env -> ExpVal
(define value-of-cdr-exp
  (lambda (exp1 env)
    (cdr (expval->pair (value-of exp1 env)))))

;; value-of-null?-exp : Exp * Env -> ExpVal
(define value-of-null?-exp
  (lambda (exp1 env)

```

```

      (let ((vall (value-of exp1 env)))
        (cases expval vall
          (emptylist-val () (bool-val #t))
          (else (bool-val #f))))))

value-of-list-exp : Listof(Exp) * Env -> ExpVal
(define value-of-list-exp
  (lambda (exps env)
    (if (null? exps)
        (emptylist-val)
        (pair-val (value-of (car exps) env)
                   (value-of-list-exp (cdr exps) env))))))

```

Exercise 3.12

```

;; the-grammar
(expression
  ("cond" (arbno expression "==>" expression) "end")
  cond-exp)

;; value-of
(cond-exp (predicates consequents)
  (letrec
    ((value-of-cond-exp
      (lambda (ps cs)
        (if (null? ps)
            (eopl:error 'value-of "undefined cond")
            (if (expval->bool (value-of (car ps) env))
                (value-of (car cs) env)
                (value-of-cond-exp (cdr ps) (cdr cs))))))
      (value-of-cond-exp predicates consequents)))

```

Exercise 3.13

ExpVal = Int + Pair + Empty – list

```

;; value-of
(if-exp (exp1 exp2 exp3)
  (let ((val1 (value-of exp1 env)))
    (if (zero? (expval->num val1))
      (value-of exp3 env)
      (value-of exp2 env))))
(zero?-exp (exp1)
  (let ((val1 (value-of exp1 env)))
    (let ((num1 (expval->num val1)))
      (if (zero? num1)
        (num-val 1)
        (num-val 0)))))
(equal?-exp (exp1 exp2)
  (let ((num1 (expval->num (value-of exp1 env)))
        (num2 (expval->num (value-of exp2 env))))
    (if (= num1 num2)
      (num-val 1)
      (num-val 0))))
(greater?-exp (exp1 exp2)
  (let ((num1 (expval->num (value-of exp1 env)))
        (num2 (expval->num (value-of exp2 env))))
    (if (> num1 num2)
      (num-val 1)
      (num-val 0))))
(less?-exp (exp1 exp2)
  (let ((num1 (expval->num (value-of exp1 env)))
        (num2 (expval->num (value-of exp2 env))))
    (if (< num1 num2)
      (num-val 1)
      (num-val 0))))
(null?-exp (exp1)
  (cases expval (value-of exp1 env)

```

```

                                (empty-list-val () (num-val 1))
                                (else (num-val 0))))

;; expval
(define-datatype expval expval?
  (num-val
    (value number?))
  (pair-val
    (val1 expval?)
    (val2 expval?))
  (empty-list-val))

```

Exercise 3.14 For simplicity we remove `cond-exp` from this interpreter. We may modify its definition to apply `value-of-bool-exp` to its predicates, but this is not ideal as we may wish for `if-exps` to be predicates as well, and those are *Expressions*.

```

;; the-grammar
(expression
  ("if" bool-exp "then" expression "else" expression)
  if-exp)

(bool-exp
  ("zero?" "(" expression ")")
  zero?-bexp)
(bool-exp
  ("equal?" "(" expression "," expression ")")
  equal?-bexp)
(bool-exp
  ("greater?" "(" expression "," expression ")")
  greater?-bexp)
(bool-exp
  ("less?" "(" expression "," expression ")")
  less?-bexp)

```

```

(bool-exp
  ("null?" "(" expression ")")
  null?-bexp)

;; value-of
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) ...)
      (var-exp (var) ...)
      (diff-exp (exp1 exp2) ...)
      (if-exp (bexp1 exp1 exp2)
        (let ((val1 (value-of-bool-exp bexp1 env)))
          (if (expval->bool val1)
              (value-of exp1 env)
              (value-of exp2 env))))))
      (let-exp (var exp1 body) ...)
      (minus-exp (exp1) ...)
      (add-exp (exp1 exp2) ...)
      (mul-exp (exp1 exp2) ...)
      (div-exp (exp1 exp2) ...)
      (cons-exp (exp1 exp2) ...)
      (car-exp (exp1) ...)
      (cdr-exp (exp1) ...)
      (empty-list-exp () ...)
      (list-exp (exps) ...))))))

;; value-of-bool-exp : Bool-exp * Env -> ExpVal
(define value-of-bool-exp
  (lambda (bexp env)
    (cases bool-exp bexp
      (zero?-bexp (exp1)

```

```

        (if (zero? (expval->num (value-of exp1 env))
            (bool-val #t)
            (bool-val #f)))
(equal?-bexp (exp1 exp2)
  (if (= (expval->num (value-of exp1 env))
        (expval->num (value-of exp2 env)))
      (bool-val #t)
      (bool-val #f)))
(greater?-bexp (exp1 exp2)
  (if (> (expval->num (value-of exp1 env))
        (expval->num (value-of exp2 env)))
      (bool-val #t)
      (bool-val #f)))
(less?-bexp (exp1 exp2)
  (if (< (expval->num (value-of exp1 env))
        (expval->num (value-of exp2 env)))
      (bool-val #t)
      (bool-val #f)))
(null?-bexp (exp1)
  (cases expval (value-of exp1 env)
    (empty-list-val () (bool-val #t))
    (else (bool-val #f)))))

```

Exercise 3.15 Prints a Scheme-readable form of an *ExpVal*.

```

;; the-grammar
(expression
  ("print" "(" expression ")")
  print-exp)

;; value-of
(print-exp (exp1)
  (begin
    (display (expval->schemeval (value-of exp1 env)))

```

```

        (newline)
        (num-val 1)))

;; expval->schemeval : ExpVal -> SchemeVal
(define expval->schemeval
  (lambda (val)
    (cases expval val
      (num-val (num) num)
      (bool-val (bool) bool)
      (pair-val (val1 val2)
        (cons (expval->schemeval val1)
              (expval->schemeval val2)))
      (empty-list-val () '()))))

```

Exercise 3.16

```

;; the-grammar
(expression
  ("let" (arbno identifier "=" expression) "in" expression)
  let-exp)

;; value-of
(let-exp (vars exps body)
  (value-of
    body
    (extend-env*
      vars
      (map (lambda (exp)
            (value-of exp env))
          exps)
      env)))

(define extend-env*
  (lambda (vars vals env)

```

```

(if (null? vars)
    env
    (extend-env
      (car vars)
      (car vals)
      (extend-env*
        (cdr vars)
        (cdr vals)
        env)))))

```

Exercise 3.17

```

;; the-grammar
(expression
  ("let*" (arbno identifier "=" expression) "in" expression)
  let*-exp)

```

```

;; value-of
(let*-exp (vars exps body)
  (letrec
    ((value-of-let*-exp
      (lambda (vars exps let*-env)
        (if (null? vars)
            (value-of body let*-env)
            (value-of-let*-exp
              (cdr vars)
              (cdr exps)
              (extend-env
                (car vars)
                (value-of (car exps) let*-env)
                let*-env))))))
    (value-of-let*-exp vars exps env)))

```

Exercise 3.18


```

;; the-grammar
(expression
  ("unpack" (arbno identifier) "=" expression "in" expression)
  unpack-exp)

;; value-of
(unpack-exp (vars lst body)
  (let ((val0 (value-of lst env)))
    (let ((vals (pair-val->list-of-expvals val0)))
      (if (= (length vars) (length vals))
          (value-of body (extend-env* vars vals env))
          (eopl:error 'value-of "bad unpack")))))

;; pair-val->list-of-expvals : ExpVal -> Listof(ExpVal)
;; expects a non-nested pair-val and returns a linear list of expvals
(define pair-val->list-of-expvals
  (lambda (val)
    (cases expval val
      (empty-list-val () '())
      (pair-val (val1 val2)
        (cons val1 (pair-val->list-of-expvals val2)))
      (else (list val)))))

```